R3.05 · TD-TP 08

PROGRAMMATION SYSTÈME

Les threads et mutex

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Le hangar à drones du LAMAE s'automatise. Chaque drone est piloté par un thread, tous partagent la même mémoire... et les mêmes ennuis. Ta mission : faire voler l'essaim sans perte de données, sans collision et sans blocage.

OBJECTIFS DE LA SÉANCE

- ☐ Créer, attendre et terminer des threads POSIX (`pthread_create`, `pthread_join`, `pthread_exit`).
- ☐ Transmettre un argument propre à chaque thread et récupérer son résultat.
- ☐ Mettre en évidence une **condition de concurrence** et la corriger par **exclusion mutuelle** (mutex).
- ☐ Reconnaître et éviter un **interblocage** (deadlock).
- ☐ Détecter les conflits d'accès avec `-fsanitize=thread` et vérifier les codes de retour des fonctions `pthread_*`.

FIL ROUGE **Opération « Essaim »**

Le hangar compte jusqu'à 26 drones, chacun piloté par un **thread**. Le chef d'escadrille (le thread `main`) les lance, les attend et fait le bilan. Chaque mission réussie te donne un **drapeau** `FLAG{...}` à noter dans `badge08.txt`. Les fichiers de départ sont dans `code/sujet08/` : `compteur.c`, `drones_args.c`, `virement.c`, `hangar_depart.c`, `vt100.h`.

Mission	Contenu	Durée
0 · Décollage	Threads et processus, premier thread	15 min
1 · Ordres de mission	Arguments et valeurs de retour	25 min
2 · Le compteur fou	Condition de concurrence, mutex, ThreadSanitizer	25 min
3 · L'étreinte fatale	Interblocage à deux mutex	15 min
4 · Le hangar	Drones rebondissants, puis n drones et collisions	40 min
Bonus	Pour les rapides : producteur / consommateur	—

COMPILATION DES PROGRAMMES MULTITHREADS

L'option `-pthread` (et non plus seulement `-lpthread`) règle à la fois la compilation **et** l'édition de liens.

TERMINAL

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -pthread prog.c -o prog
$ gcc -std=c17 -Wall -Wextra -g -pthread -fsanitize=thread prog.c -o prog_tsan
```

Comme pour les sujets précédents, les pthreads appartiennent à la norme **POSIX**, pas au C standard : avec `-std=c17`, placez `#define _POSIX_C_SOURCE 200809L` en **première ligne** de chaque programme, avant tout `#include` (indispensable notamment pour `nanosleep`).

Mission 0 — Décollage : threads et processus

Les processus sont des entités *a priori* indépendantes : chacun dispose de ses données, de son tas et de sa pile, et exécute un code non modifiable qui peut être partagé. Ils communiquent par les moyens vus précédemment (fichiers, tubes). Une autre façon de créer des activités parallèles consiste à créer des **threads**, ou **processus légers**. Linux implémente la norme POSIX : on parle de **pthreads**.

Les threads d'un même processus **partagent le même espace d'adressage** : chaque thread accède aux variables globales, au tas et aux fichiers ouverts du programme ; seuls la **pile** et les registres sont propres à chaque thread. Les threads s'exécutent **en concurrence**, et l'on retrouve les problèmes de conditions de concurrence vus en CM 03b. En contrepartie, créer un thread et passer d'un thread à l'autre coûte bien moins cher qu'avec des processus.

	Processus (`fork`)	Thread (`pthread_create`)
Mémoire	copie séparée	**partagée** (sauf la pile)
Communication	tubes, fichiers, sockets	variables partagées
Identifiant	`pid_t` (`getpid`)	`pthread_t` (`pthread_self`)
Attente de fin	`wait` / `waitpid`	`pthread_join`
Plantage d'un fils	n'affecte pas le père	**tue tout le processus**

Les fonctions de base

Un thread est identifié par une variable de type `pthread_t`. Ce type est **opaque** : on ne le suppose pas entier, on ne l'affiche pas avec `%ld`, et on compare deux identifiants avec `pthread_equal`.

```
• • •  <pthread.h>
int pthread_create(pthread_t *thread,             // identifiant du thread créé
                   const pthread_attr_t *attr,    // attributs, ou NULL par défaut
                   void *(*fn)(void *),           // fonction exécutée par le thread
                   void *arg);                    // argument passé à fn
int  pthread_join(pthread_t thread, void **retour); // attendre la fin, récupérer le retour
void pthread_exit(void *retour);                  // terminer le thread appelant (ne revient pas)
pthread_t pthread_self(void);                     // identifiant du thread appelant
int  pthread_equal(pthread_t t1, pthread_t t2);   // non nul si même thread
int  pthread_cancel(pthread_t thread);            // demander l'annulation d'un thread
```

- `pthread_create` renvoie **0** en cas de succès, sinon un **code d'erreur** (comme `EAGAIN`). Attention : contrairement aux appels système, elle ne renvoie pas -1 et ne positionne pas `errno` ; on affiche l'erreur avec `strerror(rc)`.
- Un thread se termine quand sa fonction fait `return valeur;` ou appelle `pthread_exit(valeur)`. Cette valeur est récupérée par `pthread_join`.
- Si `main` se termine (ou appelle `exit`), **tous** les threads du processus meurent. D'où l'importance de `pthread_join`.

exemple de création d'un thread

```c
#include <pthread.h>

void *tAction(void *n)
{
    /* ... travail du thread ... */
    return NULL;                // ou pthread_exit(NULL);
}

int main(void)
{
    pthread_t thread1;
    int resultat = pthread_create(&thread1, NULL, tAction, NULL);
    if (resultat != 0) { /* gérer l'erreur */ }
    pthread_join(thread1, NULL);
    return 0;
}
```

EX 1 Premier décollage

★ 10 min

Écrivez un programme dont le thread `main` affiche son PID, puis crée un thread qui reçoit la chaîne `"drone-1"` en argument et affiche `[thread] drone-1 : PID ...`. Le `main` attend la fin du thread puis affiche `pthread_equal(pthread_self(), t)`.

1. Les deux PID sont-ils identiques ? Qu'en concluez-vous ?
2. Supprimez le `pthread_join`. Que devient l'affichage du thread ? Pourquoi ?
3. Pendant qu'un programme multithread tourne (ajoutez un `sleep(30)` dans le thread), observez-le avec `ps -eLf | grep nom_du_prog` : que représente la colonne `LWP` ? et `NLWP` ?

Drapeau : `FLAG{` + valeur renvoyée par `pthread_create` en cas de succès + `}`.

> **PIÈGE CLASSIQUE** **Le thread n'est pas un fork**
> Un `exit()` appelé dans **n'importe quel** thread termine le processus entier, donc tous les threads. Pour terminer seulement le thread courant : `return` dans sa fonction, ou `pthread_exit`.

Mission 1 — Ordres de mission : arguments et résultats

EX 2 Cinq drones, cinq numéros

★★ 15 min

Compilez et exécutez plusieurs fois `drones_args.c`, qui lance 5 drones en leur passant `&i` :

drones_args.c (extrait)

```c
for (int i = 0; i < NB_DRONES; i++)
    pthread_create(&drones[i], NULL, decoller, &i);   // <-- suspect
```

1. Les numéros affichés sont-ils toujours 0, 1, 2, 3, 4 ? Pourquoi peut-on voir `5`, alors que la boucle s'arrête à 4 ?
2. Corrigez en donnant à chaque thread **sa propre** case : un tableau `ParamDrone params[NB_DRONES]` de structures contenant le numéro du drone et le nom de sa mission (`"inspection"`, `"livraison"`, ...).
3. Ajoutez le test du code de retour de chaque `pthread_create`.

INDICE

Tous les threads reçoivent **la même adresse** : celle de la variable `i` du `main`, qui continue d'évoluer pendant que les threads démarrent. C'est une condition de concurrence sur `i`. Le tableau de paramètres doit vivre au moins jusqu'au dernier `pthread_join`.

EX 3 Le rapport de mission

★★ 10 min

Un tableau global `int donnees[1000000]` contient les relevés des capteurs (`donnees[i] = i % 10`). Lancez **4 threads** ; chacun calcule la somme d'une tranche `[debut, fin[` du tableau et **renvoie** son résultat au `main` par `pthread_join`. Le `main` additionne les 4 résultats et vérifie le total (4 500 000).

SÉCURITÉ **Ne renvoyez jamais l'adresse d'une variable locale**

Quand le thread se termine, **sa pile est libérée**. Renvoyer `&somme` (variable locale du thread) donne au `main` un pointeur vers une zone détruite : comportement indéfini, résultat faux ou plantage aléatoire. Solutions : allouer le résultat avec `malloc` (le `main` fera le `free`), ou écrire le résultat dans la structure d'arguments fournie par le `main`. Même règle pour les arguments : ne transmettez pas à un thread l'adresse d'une variable locale d'une fonction qui va se terminer avant lui.

Drapeau : `FLAG{` + total obtenu + `}`.

Mission 2 — Le compteur fou : concurrence et mutex

Les **mutex** (*mutual exclusion*) servent à éviter l'incohérence des données due aux conditions de concurrence. Une **condition de concurrence** survient quand plusieurs threads accèdent à la même donnée, **dont au moins un en écriture**, sans synchronisation : le résultat dépend alors de l'ordre exact d'exécution. Le mutex **sérialise** l'accès : il protège une **section critique**, portion de code qu'un seul thread à la fois peut exécuter (exclusion mutuelle).

Sans mutex	Avec mutex
`int counter = 0;`	`pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;` `int counter = 0;`

`void functionC(void)`
`{`
`  counter++;`
`}`
	`void functionC(void)`
	`{`
	`  pthread_mutex_lock(&mutex1);`
	`  counter++;`
	`  pthread_mutex_unlock(&mutex1);`
	`}`

Exemple complet (version corrigée de l'exemple historique) :

compteur_mutex.c

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

void *functionC(void *arg);
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

int main(void)
```

```
{
    int rc1, rc2;
    pthread_t thread1, thread2;

    if ((rc1 = pthread_create(&thread1, NULL, functionC, NULL)) != 0)
        fprintf(stderr, "Thread creation failed: %s\n", strerror(rc1));
    if ((rc2 = pthread_create(&thread2, NULL, functionC, NULL)) != 0)
        fprintf(stderr, "Thread creation failed: %s\n", strerror(rc2));

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    return 0;
}

void *functionC(void *arg)
{
    (void) arg;                      // paramètre inutilisé
    pthread_mutex_lock(&mutex1);
    counter++;
    printf("Counter value: %d\n", counter);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}
```

EX 4 Le compteur fou

★ 20 min

Le fichier `compteur.c` lance deux threads qui incrémentent `n` fois la même variable globale, **sans protection**.

1. Exécutez `./compteur 1000`, puis `./compteur 1000000` et `./compteur 10000000`, plusieurs fois. Notez le nombre de mises à jour perdues. Pourquoi le problème n'apparaît-il pas (ou peu) pour de petites valeurs ?
2. Expliquez la perte : `compteur++` n'est pas **une** opération mais **trois** (lire, calculer, écrire). Vérifiez-le avec `objdump -d compteur | grep -A6 "<incrementer>"` et reconstituez un entrelacement qui perd une incrémentation (comme le compte bancaire du CM 03b).
3. Recompilez avec `-fsanitize=thread` et lancez avec `n = 1000` : lisez le rapport `data race` (fichier, ligne, threads impliqués).
4. Protégez la section critique avec un mutex. Vérifiez que le résultat est juste et que ThreadSanitizer ne signale plus rien.
5. Mesurez avec `time` : verrouiller à **chaque** incrément coûte cher. Proposez une seconde version où chaque thread compte dans une variable **locale**, puis ajoute son total au compteur global en une seule section critique. Comparez les durées.

> **PIÈGE CLASSIQUE** **Le bug qui marche 999 fois sur 1 000**
>
> Une condition de concurrence peut ne jamais se manifester sur votre machine et apparaître chez le client, sous charge, sur un processeur différent. On ne « teste » pas l'absence de concurrence : on la **démontre** par construction (section critique protégée) et on s'aide des outils (`-fsanitize=thread`, `valgrind --tool=helgrind`).

> **SÉCURITÉ** **Conditions de concurrence et TOCTOU**
>
> Les conditions de concurrence sont une vraie classe de vulnérabilités. Le cas typique est le **TOCTOU** (*Time Of Check to Time Of Use*) : on **vérifie** une condition (le solde est suffisant, la case est libre, le fichier n'existe pas), puis on **agit** ; entre les deux, un autre thread change la situation. Règle : la vérification et l'action doivent se trouver dans **la même** section critique. Vous l'appliquerez aux collisions de la mission 4.

Drapeau : `FLAG{` + le nom du rapport affiché par ThreadSanitizer (deux mots, séparés par `_`) + `}`.

Mission 3 — L'étreinte fatale : l'interblocage

Deux drones échangent de l'énergie : le drone A donne une unité à B pendant que B donne une unité à A. Chaque batterie est protégée par **son propre** mutex, et `transferer(src, dst, q)` verrouille d'abord la source, puis la destination.

EX 5 — Deadlock au hangar

★★ 15 min

1. Exécutez `virement.c` plusieurs fois. Que se passe-t-il ? (Arrêtez avec `Ctrl+C`.)
2. Lancez-le sous `gdb ./virement`, faites `run`, attendez le blocage, tapez `Ctrl+C` puis `info threads` et `thread apply all bt` : sur quelle fonction chaque thread est-il bloqué ? Dessinez le cycle « qui attend qui ».
3. Corrigez en imposant un **ordre global** de verrouillage : on verrouille toujours la batterie de plus petit `id` en premier, quel que soit le sens du transfert. Vérifiez que le total reste 2000.

INDICE

Il y a interblocage quand chaque thread **détient** une ressource et **attend** celle que détient l'autre. Les quatre conditions de Coffman (exclusion mutuelle, détention et attente, pas de réquisition, attente circulaire) sont toutes réunies ; l'ordre global de verrouillage casse l'**attente circulaire**.

PIÈGE CLASSIQUE — Oublier de déverrouiller

Un `return` au milieu d'une section critique (sur une erreur par exemple) laisse le mutex verrouillé : le prochain thread qui le demande attend pour toujours. Chaque `lock` doit avoir son `unlock` sur **tous** les chemins de sortie.

Mission 4 — Le hangar : drones rebondissants

Le terminal se pilote avec des **séquences d'échappement VT100** (fichier `vt100.h`, rappelé en annexe). Par exemple, `printf(GOTOYX, 3, 10);` place le curseur en ligne 3, colonne 10 ; les coordonnées **commencent à 1**.

EX 6 — Un drone dans le hangar

★★ 20 min

Complétez `hangar_depart.c` : un drone (un caractère) se déplace dans le hangar et **rebondit** sur les murs. Le déplacement est calculé par **un thread** ; le `main` dessine les murs, lance le thread et l'attend.

1. Dans la boucle du thread : effacer l'ancienne position, inverser `dx` (ou `dy`) si la prochaine position touche un mur, avancer, dessiner, faire une pause de 50 ms (`nanosleep`).
2. Bornez le nombre de déplacements (par exemple 400) plutôt qu'une boucle infinie : le programme doit se terminer proprement et rendre le curseur (`CURSON`).

PIÈGE CLASSIQUE — usleep est obsolète

`usleep` a été retirée de POSIX en 2008. Utilisez `nanosleep` (fonction `pause_ms` fournie). Avec `-std=c17`, il faut définir `#define _POSIX_C_SOURCE 200809L` **avant** tout `#include` pour que `nanosleep` soit déclarée.

EX 7 L'essaim et les collisions ★★★ 20 min

Passez à **n drones** (n donné en argument, 1 à 26), un thread par drone, chacun avec son symbole (`'A'`, `'B'`, ...).

1. Sans précaution, l'affichage se brouille : des caractères apparaissent au mauvais endroit. Pourquoi ? Protégez l'écran par un mutex : une séquence `GOTOYX` + caractère doit être écrite **d'un seul bloc**.
2. Gérez les collisions : une grille partagée `char grille[HAUTEUR+2][LARGEUR+2]` indique quelle case est occupée. Avant d'avancer, un drone vérifie que la case visée est libre ; sinon il change de direction au hasard.
3. Pourquoi « vérifier que la case est libre » puis « la réserver » doit-il se faire dans **la même** section critique ? Quel nom porte ce défaut s'il est mal fait ?
4. Faut-il un seul mutex (écran et grille) ou deux ? Discutez la granularité.

SÉCURITÉ Fonctions non réentrantes

`rand()` et `strtok()` utilisent un **état global caché** : appelées depuis plusieurs threads, elles produisent des conflits (et `-fsanitize=thread` les signale parfois). Utilisez leurs versions réentrantes, avec un état propre à chaque thread : `rand_r(&graine)` (une graine par drone) et `strtok_r`. De même `localtime` → `localtime_r`. La page `man 7 pthreads` liste les fonctions qui ne sont pas *thread-safe*.

Drapeau : `FLAG{` + le nom de la fonction réentrante qui remplace `rand` + `}`.

POUR LES RAPIDES Producteur / consommateur ★★★

La station radar (un thread producteur) dépose des numéros de mission dans une **file circulaire bornée** de 8 cases ; deux drones (threads consommateurs) les retirent et les traitent.

1. Protégez la file par un mutex. Un consommateur qui trouve la file vide ne doit pas boucler en testant (attente active) : il se **bloque** avec une variable de condition `pthread_cond_wait(&pasVide, &m)` ; le producteur le réveille avec `pthread_cond_signal(&pasVide)`. Symétriquement pour la file pleine.
2. Pourquoi faut-il écrire `while (nb == 0) pthread_cond_wait(...)` et non `if` ?
3. Comment prévenir les consommateurs qu'il n'y aura plus de missions ? (Idée : une valeur spéciale par consommateur.)

SÉCURITÉ pthread_cancel, à manier avec précaution

Annuler un thread de l'extérieur peut l'interrompre alors qu'il **détient un mutex** ou une ressource (mémoire allouée, fichier ouvert) : le mutex reste verrouillé à jamais et la ressource fuit. Préférez un arrêt **coopératif** : un indicateur partagé (protégé) que le thread consulte régulièrement. Si l'annulation est indispensable, voir `pthread_cleanup_push(3)`.

Annexe — Séquences VT100

vt100.h (extrait)

```c
/* VT100 : actions sur l'écran d'un terminal */
#define CLEARSCR     "\x1B[2J\x1B[;H"    // efface l'écran
#define CLEAREOL     "\x1B[K"            // efface jusqu'à la fin de la ligne
#define CLEAREOS     "\x1B[J"            // efface jusqu'à la fin de l'écran
#define CLEARLCR     "\x1B[0K"           // efface la ligne à droite du curseur
#define CLEARLCL     "\x1B[1K"           // efface la ligne à gauche du curseur
#define CLEARELN     "\x1B[2K"           // efface toute la ligne
#define CLEARCDW     "\x1B[0J"           // efface sous le curseur
#define CLEARCUP     "\x1B[1J"           // efface au-dessus du curseur
#define GOTOYX       "\x1B[%.2d;%.2dH"   // va en (ligne, colonne), à partir de 1
#define INSERTMOD    "\x1B[4h"           // mode insertion
#define OVERWRITEMOD "\x1B[4l"           // mode remplacement
#define CRLF         "\n\r"              // retour à la ligne
/* VT100 : actions sur le curseur */
#define CURSON       "\x1B[?25h"         // curseur visible
#define CURSOFF      "\x1B[?25l"         // curseur invisible
/* VT100 : actions sur les caractères affichables */
#define NORMAL       "\x1B[0m"
#define BOLD         "\x1B[1m"
#define UNDERLINE    "\x1B[4m"
#define BLINKING     "\x1B[5m"
#define INVVIDEO     "\x1B[7m"
```

demo_vt100.c

```c
#include <stdio.h>
#include "vt100.h"

int main(void)
{
    printf(CLEARSCR);
    printf(BOLD);
    printf(GOTOYX, 3, 10);
    printf("bonjour\n");
    printf(NORMAL);
    return 0;
}
```

Bilan de la séance

À RETENIR

- Les threads d'un processus partagent **tout** sauf leur pile : toute donnée partagée modifiée est une section critique potentielle.
- `pthread_*` renvoie 0 ou un **code d'erreur** (pas -1) : on le teste et on l'affiche avec `strerror`.
- Un argument propre à chaque thread ; jamais l'adresse d'une variable locale qui va disparaître.
- Vérifier **et** agir dans la même section critique (pas de TOCTOU).
- Interblocage : toujours verrouiller plusieurs mutex dans le **même ordre**.
- On détecte les conflits avec `-fsanitize=thread` ; on ne « teste » pas leur absence.

- ☐ J'ai mes 5 drapeaux dans `badge08.txt` (missions 0 à 4).
- ☐ Mes programmes compilent sans avertissement et ThreadSanitizer ne signale rien sur mes versions corrigées.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 3 pthread_create`, `man 3 pthread_join`, `man 3 pthread_mutex_lock`	Prototypes, codes d'erreur, comportements (pages installées par `manpages-posix-dev`).
`man 7 pthreads`	Vue d'ensemble : identifiants, attributs partagés ou non, fonctions *thread-safe*, implémentation NPTL.
`man 3 pthread_cond_wait`, `man 3 rand_r`, `man 3 strtok_r`	Variables de condition ; versions réentrantes.
M. Kerrisk, *The Linux Programming Interface*, ch. 29 à 33	La référence : threads, synchronisation (mutex, conditions), sûreté des threads, annulation, détails d'implémentation.
A. Tanenbaum, *Systèmes d'exploitation*	Sections critiques, sémaphores, moniteurs, interblocage ; le **dîner des philosophes**.
J.-M. Rifflet, *La programmation sous UNIX*	Référence francophone de la programmation système.
Guide ANSSI, *Règles de programmation pour le développement sécurisé de logiciels en langage C*	Bonnes pratiques de robustesse : vérification des retours, libération des ressources.
`-fsanitize=thread` (ThreadSanitizer)	Détection des *data races* et de certains interblocages à l'exécution.
`valgrind --tool=helgrind ./prog`	Détection des conflits d'accès, des erreurs d'utilisation des mutex et des ordres de verrouillage incohérents.
`gdb` : `info threads`, `thread N`, `thread apply all bt`	Examiner l'état de chaque thread (indispensable pour un interblocage).
`ps -eLf`, `top -H`, `/proc/<pid>/task/`	Observer les threads d'un processus en cours d'exécution.

Pour approfondir

> **CULTURE** **Les atomiques C11**
>
> Depuis C11, l'en-tête `<stdatomic.h>` fournit des types et opérations **atomiques** : `atomic_long compteur;` et `atomic_fetch_add(&compteur, 1);` réalisent « lire-ajouter-écrire » de façon indivisible, sans mutex. C'est idéal pour un compteur ; pour protéger plusieurs variables liées (une file, un compte et son historique), il faut toujours une section critique.

> **CULTURE** **Le dîner des philosophes**
>
> Cinq philosophes autour d'une table, cinq fourchettes, et il en faut deux pour manger. Si chacun prend d'abord la fourchette de gauche... interblocage. Ce problème classique de Dijkstra (1965) illustre les solutions vues en mission 3 (ordre global, ressource « arbitre ») et le risque de **famine**. Implémentez-le avec 5 threads et 5 mutex : c'est un excellent exercice de révision.

CULTURE **Threads et processus sous Linux**

Pour le noyau Linux, un thread est une « tâche » comme une autre, créée par l'appel système `clone` avec des options de partage (mémoire, fichiers, signaux). `fork` et `pthread_create` reposent tous deux sur `clone` : observez-le avec `strace -f ./prog`. Dans `ps -eLf`, chaque thread a son propre identifiant (`LWP`, ou TID) mais tous partagent le PID du processus.